Как объяснить PA3 и провести защиту

Что ты сделал

Ты создаёшь веб-страницу, которая ставит виртуальную колонку настоящего размера на реальную поверхность. Браузер получает положение телефона и найденной поверхности от системы AR. Three.js строит и рисует колонку. Масштаб задан геометрией: один метр в коде соответствует одному метру в пространстве WebXR.

Твой вариант: **242458 → 8, 5, 4, 2 → колонка, single, 12°, лицом к пользователю**. Размеры: ширина 25 см, глубина 30 см, высота 100 см. Два круглых динамика находятся на стороне +Z. Начало координат модели стоит в центре основания: поэтому нижняя грань попадает на поверхность без дополнительного смещения на половину высоты.

Главная мысль для защиты: приложение должно согласовать три вещи — координаты телефона, координаты поверхности и координаты модели. Хороший внешний вид ещё не доказывает правильную привязку или масштаб.

Кто за что отвечает

Система AR устройства анализирует изображения и датчики движения. Она оценивает позу телефона и геометрию окружения. **WebXR** передаёт приложению разрешённые позы, результаты hit test и anchors. **Three.js** создаёт геометрию, материалы, освещение и выводит сцену в XR-слой. JavaScript страницы управляет жизненным циклом, фильтром наклона и взаимодействиями.

Не говори, что твой JavaScript самостоятельно реализует компьютерное зрение. Он пользуется результатами ARCore или соответствующей системы устройства.

Viewer и local на понятном примере

Представь, что ты стоишь перед колонкой. В комнате она находится в одной точке. Но относительно твоего лица она сначала впереди, а когда ты обойдёшь её — сбоку или сзади. Это одна физическая точка в двух системах координат.

viewer двигается вместе с телефоном. Поэтому луч поиска поверхности задаётся в viewer: куда смотрит устройство, туда направлен центральный луч. **local** остаётся системой координат сцены в окрестности начала сессии. Результат попадания читается в local, чтобы положение колонки можно было согласовать с положением камеры. Local не является географической системой и не обещает идеальную неподвижность при уточнении карты.

local-floor меняет высоту начала координат так, чтобы y=0 соответствовал полу, который система определила или оценила. Он не добавляет более точный поиск поверхностей. Попадание на стол остаётся попаданием на стол; смена начала координат не превращает стол в пол.

Hit test и anchor

Hit test отвечает на вопрос: где луч пересёк реальную геометрию в этом кадре? Возвращённый hit содержит позу попадания. Само получение массива hits не создаёт устойчивые пространственные anchors.

Anchor — отдельный объект, через который система уточняет положение выбранной привязки по мере уточнения карты. После создания anchor каждый кадр берётся его новая поза. Если просто сохранить однажды полученную матрицу, модель останется на этих числах local-координат, даже когда система пересмотрит оценку пространства.

Не обещай нулевой дрейф. С anchor всё равно возможны потеря трекинга, изменение оценки и погрешность. В этой работе anchors не сохраняются между сессиями. **anchor.delete()** освобождает привязку при перемещении, Clear и завершении.

Как читать матрицу

Матрица имеет размер 4×4. В массиве Three.js она хранится по столбцам. Положение находится в элементах 12, 13, 14. Ось Y поверхности, то есть нормаль, — в элементах 4, 5, 6. Нельзя читать их как вторую строку напечатанного массива.

Нормаль n сравнивается с мировой вертикалью u=(0,1,0). Косинус угла: n·u / |n| = n.y / |n|. Затем acos переводится из радиан в градусы. Ограничение косинуса диапазоном [-1,1] предотвращает NaN из-за округления. При угле не больше 12° кольцо зелёное; при большем — красное, установка запрещена. Направленная вниз нормаль имеет 180°, а не 0°.

Учебный расчёт, не эксперимент: n=(0,0.978148,0.207912), |n|≈1; acos(0.978148)≈12°. Для отчёта нужно повторить вычисление с настоящими 16 значениями из Capture pose.

Почему нажатие только ставит флаг

Событие **select** сообщает о действии пользователя. Положение нужно брать из актуального XRFrame. Поэтому обработчик устанавливает **placeRequested=true**, а следующий кадр заново проверяет трекинг и наклон и создаёт размещение. Нельзя хранить XRHitTestResult и вызывать его методы когда-нибудь после окончания кадра.

Вызов **hit.createAnchor()** делается внутри кадра, хотя его Promise завершится позже. Пользователь за это время может нажать Clear или STOP. У записи есть **alive**, а у сессии — номер поколения. Если ответ относится к уже удалённой записи или старой сессии, новый anchor сразу удаляется. Иначе после Clear могла бы вернуться невидимая, но продолжающая жить привязка.

Как колонка поворачивается

Передняя сторона модели — +Z. Из точки установки к положению телефона берутся смещения dx и dz. Угол yaw = atan2(dx,dz). Высота телефона не участвует, поэтому колонка не наклоняется к лицу.

При привязке к anchor сначала вычисляется относительный поворот **inverse(R_hit) * R_yaw**. Затем каждый кадр используется **M_anchor * R_offset**. Это сохраняет выбранное направление относительно привязки. Колонка поворачивается к человеку в момент установки; она не должна постоянно следить за ним при обходе.

Карта функций

Функция	Что объяснить преподавателю
variantFromId	Как четыре цифры задают правила
buildProduct	Примитивы, метры, центр основания, проверка Box3
loadScene	WebGL2, параметры renderer, свет, кольцо, controller
detectSupport	Почему unsupported показывает сообщение
onRequestSession	Пользовательский жест и required/optional features
onSessionStarted	setSession, local/viewer, один hit-test source
onVisibilityChange	Пауза без кадров, сброс старого ввода, повторный поиск после возврата
render	Проверка трекинга, текущий hit, угол, запрос установки, anchors
placeProduct	Single/multi, yaw, createAnchor в активном кадре
removeEntry	Удаление модели и anchor
onSessionEnd	Общее завершение для STOP и системного выхода

Что происходит при системной паузе

Системное меню или запрос разрешения может временно убрать AR из фокуса. В состоянии hidden браузер вообще может перестать присылать XR-кадры. Поэтому одного условия внутри render недостаточно: обработчик visibilitychange сразу скрывает кольцо и предметы и очищает запросы установки и снимка позы.

После возвращения приложение переходит в recovering. Оно ждёт настоящую позу камеры и новое попадание на поверхность, показывает новое кольцо и только затем принимает новое нажатие. Это предотвращает неожиданную установку из старого запроса. Сама привязка при паузе не удаляется: при восстановлении её положение снова читается из anchorSpace.

Прерванный паузой опыт E4 получает статус interrupted. Нельзя записывать время, проведённое в системном меню, как время поиска поверхности. Для повторного измерения запусти новую пробу.

Три исправления кода из книги

Первое: нужен WebGL2 и объект параметров renderer с **context: gl**. Второе: старые имена вроде **RingBufferGeometry** заменены современными **RingGeometry**. Третье: hit-test source запрашивается один раз при старте сессии, а не каждый кадр.

В проверенном npm-исходнике 0.186.1 **setSession** начинается со строки 410. Он обеспечивает совместимость GL, настраивает XR-слой, запрашивает выбранное пространство и запускает XR-анимацию. Свой **setupWebGLLayer()** дублировал бы работу Three.js. Номера строк r186 и npm 0.186.1 различаются — в отчёте указаны строки именно установленного пакета.

Эксперименты без выдуманных результатов

Сначала найди Android с ARCore или Quest 3/3S. На iPhone эту работу по требованиям не продемонстрировать. Запиши модель, версию ОС и браузера. Подключи устройство через ADB port forwarding либо открой HTTPS-страницу. Эмулятор полезен для разработки, но не заменяет устройство в видео.

E1. До движения запиши ожидаемое изменение local-координат. В AR нажми Capture pose; пройди около 1 м вперёд и нажми ещё раз. Сравни векторы камеры и две матрицы одного hit. Не требуй изменения именно Z=-1: направление зависит от начальной ориентации local.

E2. Отметь основание колонки на полу малярной лентой. В режиме anchors on поставь модель, отойди на 3 м и вернись по тому же пути. Измерь расхождение основания с лентой. Повтори в новой сессии с anchors off, при похожем освещении и маршруте. Сравни сантиметры, а не только local-координаты: anchor как раз может менять численную позу ради сохранения физической привязки. Если anchors недоступны, так и запиши; эксперимент on/off на таком устройстве невозможен.

E3. Сними зелёное кольцо и красное кольцо на наклонённой устойчивой книге или доске. На каждом снимке должна быть панель с ID и углом. Для одного размещения экспортируй матрицу и посчитай угол вручную. Красное нажатие не должно менять счётчик.

E4. Выбери поверхность перед START AR. Подготовь три поверхности: с рисунком, белую матовую и стеклянную/глянцевую. Для каждой проведи три новые сессии при одинаковом освещении и сходном расстоянии. Таймер начинается при готовности hit-test source. Запиши время до первого hit; за 20 секунд без hit фиксируй неудачу, а не число 20 как успешный результат. Новая сессия браузера не обязательно полностью очищает внутреннюю карту ARCore; укажи это как ограничение.

E5. Поставь колонку у вертикальной метровой рейки или рулетки в той же плоскости глубины. Запиши наблюдаемую высоту. Ошибка со знаком = наблюдаемая высота в см минус 100; абсолютная ошибка = модуль этого числа. Размер Box3 не заменяет эту проверку. Назови минимум две причины ошибки: оценка поверхности/трекинга и неточное физическое сравнение или перспектива.

E6. Возьми два действительно наблюдавшихся сбоя. Для каждого объясни, почему покупатель казахстанского магазина может отказаться от покупки и какое изменение интерфейса поможет. Не утверждай, что приложение получает весь видеопоток или полную карту комнаты: текущие функции дают позы и hits. Доступ к raw camera — отдельная возможность, которая здесь не запрашивается.

В конце сессий выйди из AR и скачай JSON. Сохрани оригиналы снимков и видео. В **measurements.json** перенеси настоящие числа и пути к снимкам; пустые значения означают, что опыт ещё не выполнен.

Сценарий непрерывного видео на 75 секунд

Время	Действие
0–8 с	Показать страницу с ID, нажать START AR
8–20 с	Найти зелёное кольцо; направить на наклонённую доску, показать красное
20–30 с	Вернуться на допустимую поверхность и поставить колонку
30–42 с	Нажать в другом месте: колонка переместится, количество останется 1
42–58 с	Обойти колонку и показать её сбоку
58–65 с	Нажать Clear; счётчик 0, новая колонка не появляется
65–75 с	STOP, затем START AR повторно

Запись делается системным рекордером самого AR-устройства, без монтажных склеек. Если поверхность не находится вовремя, пересними весь дубль. Не пытайся подменить реальный AR-сеанс вращающейся моделью на компьютере.

Изменения вживую

В начале **index.js** есть SETTINGS. Для нового порога измени **tolerance**. Для смены режима поставь **mode: 'multi'** или **'single'**. Для лимита три установи **capacity: 3**. Для поворота от пользователя установи **faceAway: true**. После изменения перезагрузи страницу и запусти новую сессию. Исходный вариант для сдачи восстанови.

Подписи варианта, подсказки и счётчик собираются из SETTINGS. После изменения настроек и перезагрузки страницы интерфейс показывает фактический режим, порог и правило поворота. Отдельно менять эти подписи в HTML уже не нужно. Продукт и ID для сдачи остаются твоими.

Если просят скрывать кольцо при потере трекинга, покажи начало render: сначала **reticle.visible=false**, затем попадание обрабатывается только при **tracked=true**. **emulatedPosition** означает, что полноценная позиция не отслеживается, поэтому установка тоже запрещена.

Вопросы, на которые надо отвечать уверенно

Почему два света? HemisphereLight даёт мягкое освещение сверху и снизу. DirectionalLight помогает увидеть объём. Это фиксированное освещение модели, а не оценка реального света комнаты.

Почему красное кольцо всё равно видно? Система нашла поверхность, но её наклон нарушает правило. Это другой случай, чем отсутствие hit, когда кольца нет вообще.

Почему Clear не создаёт ещё одну модель? beforexrselect подавляет XR-select над панелью. DOM click всё равно выполняет действие Clear.

Почему нужно HTTPS? WebXR — API с доступом к пространственному трекингу и разрешениям устройства; браузер требует secure context. Localhost считается доверенным локальным контекстом.

Можно ли назвать тесты доказательством хорошего трекинга? Нет. Тесты показывают правильность логики на заданных входных данных. Дрейф, поверхности и масштаб проверяются только на устройстве.

Короткое выступление на английском

My project is a true-scale floor-speaker preview. My student ID is 242458. Its last four digits give me a speaker, single placement, a twelve-degree surface tolerance and a front that faces the user at placement.

The application requests a viewer-space hit-test source once per session. I read each hit in local space and use the Y column of its pose matrix as the surface normal. A valid surface produces a green reticle. A red reticle cannot place a product.

The select handler only records an intention. Placement happens in the next XR frame using that frame's hit. When anchors are enabled, I create an anchor and update the product from its pose every frame. Replacing the speaker deletes the old anchor. Clear and both session-exit paths release the same resources.

The geometry measures twenty-five by thirty by one hundred centimetres. That verifies the model dimensions. Physical scale accuracy and tracking drift still need device measurements; I do not treat the desktop preview as evidence of either.

Что осталось до полной сдачи

Нужны настоящий AR-сеанс, реальные E1–E5, два наблюдения для E6, скриншоты с панелью и demo.mp4. Затем обнови PDF и README реальной моделью устройства и версиями. Нужны минимум пять содержательных PA3-коммитов за два разных дня и финальный ZIP из последнего своевременного коммита. Дедлайн: 28 октября 2026, 22:00 по времени Астаны.

Документ задания прямо запрещает генерировать сдаваемый код и отчёт ИИ. Этот материал поможет разобраться в реализации и подготовиться, но разрешение на такое использование в оцениваемой индивидуальной работе может дать только преподаватель.